HELLA OPS — Functions 구현 우선순위 + 파일 구조 + 함수별 Step 분해표

---

0. 문서 목적

이 문서는 **HELLA company 업무관리사이트**의 Firebase Functions를 실제로 구현하기 직전, 어떤 순서로 만들고, 어떤 파일 구조로 배치하고, 각 함수를 어떤 초원자 단위로 쪼개서 개발해야 하는지까지 완전히 고정하는 **실행 전 최종 작업 설계서**다.

이 문서는 아래 3개 문서의 다음 단계 문서다.

1. Atomic PRD
2. Firestore Rules 초원자 설계서
3. Functions 초원자 설계서

즉, 이 문서는 개념 문서가 아니라 **실제 개발 순서와 작업 단위를 고정하는 실전 실행 문서**다.

---

1. 이 문서가 해결하는 문제

Functions 설계가 아무리 좋아도, 아래 4가지가 없으면 실제 개발 중 반드시 흔들린다.

1. 무엇부터 먼저 만들지 우선순위가 흔들린다.
2. 파일 구조가 도메인별로 정리되지 않아 나중에 꼬인다.
3. 한 번에 너무 큰 단위를 구현하다가 에이전트가 표류한다.
4. 검증 기준이 없어서 "만든 것처럼 보이는 코드"가 생긴다.

따라서 이 문서는 다음을 고정한다.

- 구현 우선순위
- 파일/폴더 구조
- 함수별 개발 순서
- 함수별 세부 Step
- Step별 PASS/FAIL 기준
- 의존 관계
- 개발 시작 기준

---

2. 최상위 구현 원칙

2.1 절대 원칙

- Functions는 **상태를 직접 다루는 유일한 서버 실행 계층**이다.
- 상태 전이는 UI가 아니라 Functions가 결정한다.

- 로그 없는 상태 변경은 실패로 간주한다.
- 교차 컬렉션 정합성이 필요한 작업은 반드시 트랜잭션으로 묶는다.
- Functions 구현은 화면 순서가 아니라 **무결성 우선 순서**로 진행한다.

2.2 구현 우선순위 기준

개발 순서는 아래 우선순위를 따른다.

1. **보안/인증 공통부**
2. **상태 전이 핵심 엔진**
3. **로그/감사 추적**
4. **장비/NFC 트랜잭션**
5. **일정/현장 작업 흐름**
6. **운영 보조/집계 함수**
7. **차기 확장 함수**

즉, 대시보드 재집계보다 먼저 만들어야 하는 것은 `recordWorkStart`, `submitClosing`, `checkoutEquipment` 같은 **핵심 상태 함수**다.

---

3. 전체 구현 우선순위 (확정)

Priority 0 — 공통 기반

이 단계는 모든 Function이 공통으로 사용하는 기반 계층이다. 이 단계가 없으면 이후 함수에서 인증/에러/검증/로그 구조가 중복된다.

구현 대상:

- shared/auth.ts
- shared/errors.ts
- shared/firestore.ts
- shared/logger.ts
- shared/validators.ts
- shared/timestamps.ts
- shared/audit.ts
- shared/response.ts
- shared/roles.ts
- shared/transaction.ts

이 단계는 **무조건 가장 먼저** 구현한다.

Priority 1 — 일정 생성/배정 엔진

이 단계는 관리자 운영의 출발점이다. 일정이 생성되고 사원이 배정되어야, 사원 작업 시작/종료/마감 흐름도 개발 테스트가 가능하다.

구현 대상:

- createScheduleWithWorkers
- updateScheduleAndWorkers
- cancelSchedule

이 단계 완료 후, 관리자가 일정을 만들고 사원을 배정하는 구조가 완성되어야 한다.

Priority 2 — 사원 작업 시작/종료 엔진

사원 현장 흐름의 핵심 1차 엔진이다. 마감 제출보다 먼저 시작/종료 기록 구조가 정확히 잡혀야 한다.

구현 대상:

- recordWorkStart
- recordWorkEnd
- recomputeScheduleStatus

이 단계 완료 후, `schedule_workers`와 `schedules` 간 상태 전이 기본 흐름이 맞아야 한다.

Priority 3 — 마감 제출/검토 엔진

현장 기록의 공식 확정 흐름이다. 사진, 특이사항, 시작/종료, closing 상태, schedule 완료 판정이 묶인다.

구현 대상:

- submitClosing
- syncSchedulePhotoFlags
- reviewClosing

이 단계 완료 후, 사원이 현장 작업을 종료하고 공식 마감 제출까지 가능해야 한다.

Priority 4 — 장비/NFC 메타 엔진

실제 NFC 반출/반입 전에, 장비와 태그를 등록하고 안전하게 매핑하는 구조가 먼저 필요하다.

구현 대상:

- createEquipment
- updateEquipmentMeta
- assignNfcTag

이 단계 완료 후, 장비 등록과 NFC 연결까지 완료되어야 한다.

Priority 5 — 장비 반출/반입 핵심 엔진

이 프로젝트의 고위험 핵심 기능이다. NFC, 상태 전이, 로그, 트랜잭션, 동시성 방어가 모두 걸려 있다.

구현 대상:

- checkoutEquipment
- checkinEquipment
- forceUpdateEquipmentStatus
- markEquipmentOverdueBatch

이 단계 완료 후, NFC 기반 장비 흐름이 실제 사용 가능한 수준이 되어야 한다.

Priority 6 — 정기청소 참여 엔진

MVP 범위 안에 있지만, 핵심 운영 축은 일정/마감/NFC보다 우선순위가 낮다.

구현 대상:

- createRegularCleaningAttendanceBatch
- recordRegularCleaningStart
- recordRegularCleaningEnd
- approveRegularCleaningAttendance

Priority 7 — 운영/모니터링/로그 보조 엔진

운영 안정화 단계에서 붙는 함수들이다. 핵심 기능이 먼저 안정화된 후 들어간다.

구현 대상:

- createDownloadLog
- recalcDashboardMetrics
- detectDelayedSchedulesBatch
- cleanupDrafts
- upsertUserProfile
- setUserActiveState
- setUserRole

주의: 사용자 관리 함수는 중요하지만, 초기 개발 시작 시점에서는 로그인/권한 골격과 UI 개발이 먼저 가능한 정도만 확보하면 된다. 실제 완성도 기준으로는 Priority 7이 아니라 운영 단계에서 병행 가능하다.

---

4. 최종 구현 순서 (고정본)

아래 순서를 기준으로 개발한다.

Phase F0 — Shared 공통부

1. shared/errors.ts
2. shared/response.ts
3. shared/logger.ts
4. shared/firestore.ts
5. shared/timestamps.ts
6. shared/roles.ts
7. shared/auth.ts
8. shared/validators.ts
9. shared/audit.ts
10. shared/transaction.ts

Phase F1 — 일정 생성/배정

1. createScheduleWithWorkers.ts
2. updateScheduleAndWorkers.ts
3. cancelSchedule.ts

Phase F2 — 사원 시작/종료

1. recordWorkStart.ts
2. recordWorkEnd.ts
3. recomputeScheduleStatus.ts

Phase F3 — 마감 엔진

1. syncSchedulePhotoFlags.ts
2. submitClosing.ts
3. reviewClosing.ts

Phase F4 — 장비 메타/NFC 등록

1. createEquipment.ts
2. updateEquipmentMeta.ts
3. assignNfcTag.ts

Phase F5 — 장비 반출/반입

1. checkoutEquipment.ts
2. checkinEquipment.ts
3. forceUpdateEquipmentStatus.ts
4. markEquipmentOverdueBatch.ts

Phase F6 — 정기청소

1. createRegularCleaningAttendanceBatch.ts
2. recordRegularCleaningStart.ts
3. recordRegularCleaningEnd.ts
4. approveRegularCleaningAttendance.ts

Phase F7 — 운영 보조

1. createDownloadLog.ts
2. recalcDashboardMetrics.ts
3. detectDelayedSchedulesBatch.ts
4. cleanupDrafts.ts
5. upsertUserProfile.ts
6. setUserActiveState.ts
7. setUserRole.ts

---

5. 실제 파일 구조 (최종 권장본)

```
functions/
 package.json
 tsconfig.json
 .eslintrc.cjs
 .gitignore
 src/
  index.ts
  config/
   region.ts
   env.ts
   constants.ts
  shared/
   auth.ts
   roles.ts
   errors.ts
   response.ts
   logger.ts
   firestore.ts
   validators.ts
   timestamps.ts
   audit.ts
   transaction.ts
   fieldMasks.ts
   ids.ts
   statusGuards.ts
  types/
   auth.ts
   common.ts
   schedule.ts
   equipment.ts
   closing.ts
   logs.ts
   maintenance.ts
  domains/
   schedules/
    createScheduleWithWorkers.ts
```

```
    updateScheduleAndWorkers.ts
    cancelSchedule.ts
    recordWorkStart.ts
    recordWorkEnd.ts
    submitClosing.ts
    reviewClosing.ts
    recomputeScheduleStatus.ts
    syncSchedulePhotoFlags.ts
    utils/
      scheduleValidators.ts
      scheduleReaders.ts
      scheduleCounters.ts
      scheduleStatusCalculator.ts
      scheduleWriteBuilders.ts
  equipments/
    createEquipment.ts
    updateEquipmentMeta.ts
    assignNfcTag.ts
    checkoutEquipment.ts
    checkinEquipment.ts
    forceUpdateEquipmentStatus.ts
    markEquipmentOverdueBatch.ts
    utils/
      equipmentValidators.ts
      equipmentReaders.ts
      equipmentStatusCalculator.ts
      equipmentWriteBuilders.ts
      nfcNormalizer.ts
  regularCleaning/
    createRegularCleaningAttendanceBatch.ts
    recordRegularCleaningStart.ts
    recordRegularCleaningEnd.ts
    approveRegularCleaningAttendance.ts
    utils/
      regularCleaningValidators.ts
      regularCleaningReaders.ts
  users/
    upsertUserProfile.ts
    setUserActiveState.ts
    setUserRole.ts
    utils/
      userValidators.ts
      userReaders.ts
  logs/
    createDownloadLog.ts
  maintenance/
    recalcDashboardMetrics.ts
    detectDelayedSchedulesBatch.ts
    cleanupDrafts.ts
    utils/
      dashboardReaders.ts
```

```
    delayedScheduleDetector.ts
  __tests__/
  shared/
  schedules/
  equipments/
  regularCleaning/
  users/
  maintenance/
```

---

6. index.ts 노출 정책

`index.ts` 는 단순 export 집합체로 유지한다. 모든 비즈니스 로직은 각 도메인 파일 안에 있어야 하며, `index.ts` 는 진입점 역할만 한다.

권장 예시:

```
export { createScheduleWithWorkers } from './domains/schedules/createScheduleWithWorkers'
export { updateScheduleAndWorkers } from './domains/schedules/updateScheduleAndWorkers'
export { cancelSchedule } from './domains/schedules/cancelSchedule'

export { recordWorkStart } from './domains/schedules/recordWorkStart'
export { recordWorkEnd } from './domains/schedules/recordWorkEnd'
export { submitClosing } from './domains/schedules/submitClosing'
export { reviewClosing } from './domains/schedules/reviewClosing'
export { recomputeScheduleStatus } from './domains/schedules/recomputeScheduleStatus'
export { syncSchedulePhotoFlags } from './domains/schedules/syncSchedulePhotoFlags'

export { createEquipment } from './domains/equipments/createEquipment'
export { updateEquipmentMeta } from './domains/equipments/updateEquipmentMeta'
export { assignNfcTag } from './domains/equipments/assignNfcTag'
export { checkoutEquipment } from './domains/equipments/checkoutEquipment'
export { checkinEquipment } from './domains/equipments/checkinEquipment'
export { forceUpdateEquipmentStatus } from './domains/equipments/forceUpdateEquipmentStatus'
export { markEquipmentOverdueBatch } from './domains/equipments/
markEquipmentOverdueBatch'
```

---

7. 공통 파일별 책임 정의

7.1 shared/auth.ts

책임:

- callable context 인증 확인
- request.auth 검증

- users/{uid} 조회
- isActive 검증
- AuthContextPayload 생성

절대 넣지 말 것:

- 비즈니스 상태 전이 로직
- schedule/equipment 전용 계산

7.2 shared/errors.ts

책임:

- 표준 에러 클래스
- code/message/retryable 구조 정의
- HttpsError 매핑

7.3 shared/response.ts

책임:

- 성공 응답 포맷 통일
- ok/code/message/data 구조 생성

7.4 shared/logger.ts

책임:

- 서버 로그 포맷 통일
- info/warn/error 레벨 래퍼
- 민감정보 마스킹 정책

7.5 shared/firestore.ts

책임:

- db 인스턴스
- 공통 collection ref builder
- 안전한 get/set/update wrapper

7.6 shared/validators.ts

책임:

- 문자열/배열/필수값/enum/시간 입력 검증
- 공통 validator 제공

7.7 shared/timestamps.ts

책임:

- 서버 타임스탬프 래퍼
- ISO 변환 도우미
- 시간 비교 helper

7.8 shared/audit.ts

책임:

- audit log 문서 payload 생성
- 장비/일정/권한 변경 로그 표준화

7.9 shared/statusGuards.ts

책임:

- 상태 전이 허용 여부 검사
- schedules / schedule_workers / schedule_closings / equipments 전이 검증

이 파일은 매우 중요하다. 상태 전이 검증 로직이 각 함수 파일에 중복되면 나중에 drift가 생긴다.

7.10 shared/transaction.ts

책임:

- runTransaction 공통 래퍼
- 충돌 시 표준 에러 변환
- 재시도 정책 래퍼

---

8. 함수별 구현 난이도 및 선행 의존성

함수	난이도	선행 필요	비고
createScheduleWithWorkers	높음	auth, validators, audit, transaction	일정/배정 동시 생성
updateScheduleAndWorkers	매우 높음	createScheduleWithWorkers	차집합 처리
cancelSchedule	중간	auth, audit, transaction	상태 취소
recordWorkStart	높음	auth, statusGuards, transaction	schedule 연동
recordWorkEnd	높음	recordWorkStart	종료 검증
submitClosing	매우 높음	recordWorkEnd, syncSchedulePhotoFlags	사진/상태/마감 통합
reviewClosing	중간	submitClosing	관리자 검토
recomputeScheduleStatus	매우 높음	scheduleReaders, statusGuards	핵심 계산기
syncSchedulePhotoFlags	중간	photo metadata 규칙	캐시 동기화
createEquipment	중간	auth, audit	장비 등록
updateEquipmentMeta	중간	createEquipment	상태 필드 수정 금지
assignNfcTag	높음	createEquipment, transaction	중복 ACTIVE 방지
checkoutEquipment	매우 높음	assignNfcTag, transaction, audit	핵심 NFC 엔진
checkinEquipment	매우 높음	checkoutEquipment	반입 엔진
forceUpdateEquipmentStatus	높음	createEquipment	관리자 예외 처리
markEquipmentOverdueBatch	중간	checkoutEquipment	배치
createRegularCleaningAttendanceBatch	중간	auth, validators	반복 생성
recordRegularCleaningStart	낮음	auth	시작 입력
recordRegularCleaningEnd	낮음	recordRegularCleaningStart	종료 입력
approveRegularCleaningAttendance	중간	관리자 권한	승인
recalcDashboardMetrics	중간	readers	운영 캐시
detectDelayedSchedulesBatch	높음	recomputeScheduleStatus	지연 판정

함수	난이도	선행 필요	비고
cleanupDrafts	낮음	draft 전략 확정 후	확장

---

9. 함수별 Step 분해표 — 공통 템플릿

모든 Function은 아래 8단계 템플릿으로 개발한다.

공통 Step 템플릿

Step A. 타입 정의

- 입력 타입
- 출력 타입
- 내부 payload 타입

Step B. 인증/권한 가드

- auth 확인
- isActive 확인
- role 확인

Step C. 입력값 검증

- 필수값
- enum
- 시간
- trim
- 중복/길이 정책

Step D. 참조 문서 조회

- schedule/equipment/user 등 필수 문서 조회
- 존재 여부 확인

Step E. 상태 전이 검증

- 이전 상태 확인
- 허용 전이 여부 확인

Step F. 트랜잭션/쓰기 처리

- 실제 write 수행
- 관련 컬렉션 동시 갱신
- cache 필드 갱신

Step G. 로그 생성

- equipment_logs

- audit_logs
- download_logs

Step H. 응답 반환 및 테스트

- 응답 payload
- emulator 테스트
- PASS/FAIL 판정

---

10. 함수별 Step 분해표 — 일정 도메인

10-1. createScheduleWithWorkers Step 분해

Step 1

입력 타입 정의

- CreateScheduleInput
- CreateScheduleResult

Step 2

관리자 인증 가드 구현

- ADMIN / SUPER_ADMIN만 허용

Step 3

필수 입력 validator 구현

- 제목
- 주소
- 날짜
- 시작시간
- workerIds

Step 4

카테고리/하위분류 활성 검증 구현

Step 5

사원 유효성 검증 구현

- users 존재

- isActive
- 중복 제거
- leaderUserId 포함 여부

Step 6

트랜잭션 write builder 구현

- schedule payload
- schedule_worker payload 배열

Step 7

트랜잭션 구현

- schedule 생성
- worker docs 생성
- cache 기록
- audit log 생성

Step 8

Emulator 테스트

- 정상 생성
- 비활성 사원 포함 실패
- 잘못된 시간 실패
- category 비활성 실패

PASS 기준

- schedule 1개 + worker n개 생성
- 기본 상태값 정확
- audit log 생성
- 실패 시 partial write 없음

10-2. updateScheduleAndWorkers Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증 검증

Step 3

기존 schedule 조회

Step 4

수정 가능 상태 검증

- COMPLETED/CANCELLED 수정 제한 정책 적용

Step 5

기존 workerIds와 신규 workerIds 차집합 계산 유틸 구현

Step 6

추가 인원 생성 builder 구현

Step 7

제거 인원 CANCELLED 처리 builder 구현

Step 8

schedule 본문 업데이트 builder 구현

Step 9

트랜잭션 실행

Step 10

audit log 생성

Step 11

테스트

- 단순 메타 수정
- worker 추가
- worker 제거
- STARTED worker 제거 시 정책 확인

PASS 기준

- 기존 참여 이력 보존
- STARTED/ENDED 기록 손상 없음

- assignmentStatus 기반 제거 처리

10-3. cancelSchedule Step 분해

Step 1

입력 타입 + 사유 필수 validator

Step 2

관리자 인증

Step 3

기존 상태 조회 및 중복 취소 차단

Step 4

취소 가능한 worker 범위 계산

Step 5

schedule CANCELLED write builder 구현

Step 6

관련 worker CANCELLED 처리

Step 7

audit log 생성

Step 8

테스트

- 정상 취소
- 이미 취소된 일정 재취소 실패

PASS 기준

- schedule.status = CANCELLED
- cancelledAt/reason 기록
- 관련 worker 정리

10-4. recordWorkStart Step 분해

Step 1

입력 타입 정의

Step 2

사원 인증/활성 확인

Step 3

본인 schedule_worker 조회

Step 4

본인 배정 여부 확인

Step 5

일정 상태 검증

- CANCELLED/COMPLETED 차단

Step 6

START 전이 허용 검사

Step 7

actualStartAt 결정

- manual 입력 or server now

Step 8

트랜잭션 구현

- schedule_worker STARTED
- startedWorkerCount 계산
- schedule.status 재계산

Step 9

audit log 생성

Step 10

테스트

- 정상 시작
- 중복 시작 차단
- 타인 일정 차단
- 취소 일정 차단

PASS 기준

- schedule_worker STARTED
- actualStartAt 기록
- 필요 시 schedule IN_PROGRESS 전이

10-5. recordWorkEnd Step 분해

Step 1

입력 타입 정의

Step 2

사원 인증/배정 확인

Step 3

STARTED 상태 확인

Step 4

actualStartAt 존재 확인

Step 5

actualEndAt 검증

- start 이후인지 확인

Step 6

트랜잭션 구현

- workStatus ENDED
- ended count 갱신

Step 7

audit log 생성

Step 8

테스트

- 정상 종료
- 시작 없이 종료 실패
- 역시간 종료 실패
- 중복 종료 차단

PASS 기준

- actualEndAt 기록
- workStatus ENDED
- partial write 없음

10-6. submitClosing Step 분해

Step 1

입력 타입 정의

- beforePhotoIds
- afterPhotoIds
- notes
- draftMode

Step 2

사원 인증/배정 확인

Step 3

schedule_worker / schedule / existing closing 조회

Step 4

시작/종료 시간 존재 검증

Step 5

photo metadata 조회 및 소유권 검증

Step 6

before/after 최소 수 검증

Step 7

FAILED 업로드 존재 여부 검증

Step 8

closing payload builder 구현

Step 9

photo flag sync 로직 호출

Step 10

schedule_worker CLOSED 후보 처리

Step 11

schedule COMPLETED 여부 계산

Step 12

트랜잭션 커밋

Step 13

audit log 생성

Step 14

테스트

- 정상 제출
- 사진 부족 실패
- FAILED 사진 실패
- 중복 제출 실패
- start/end 없음 실패

PASS 기준

- closing SUBMITTED
- photo counts 정확
- schedule flags 동기화

- 조건 충족 시 schedule COMPLETED

10-7. reviewClosing Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증

Step 3

closing 상태 조회

Step 4

허용 상태 전이 검사

Step 5

REVIEW / APPROVE / REJECT / FORCE_ADJUST 처리 분기 구현

Step 6

관리자 메모/승인자/사유 기록

Step 7

audit log 생성

Step 8

테스트

- 정상 승인
- 잘못된 전이 차단

PASS 기준

- closing next status 정확
- approvedBy / approvedAt 등 부가 필드 정확

10-8. recomputeScheduleStatus Step 분해

Step 1

입력 타입 정의

Step 2

schedule 조회

Step 3

관련 schedule_workers 조회

Step 4

관련 closings 집계

Step 5

status calculator 구현

- PLANNED
- IN_PROGRESS
- COMPLETED
- DELAYED
- CANCELLED 유지

Step 6

cache field calculator 구현

Step 7

필요 시 schedule 업데이트

Step 8

테스트

- 일부만 시작된 경우 IN_PROGRESS
- 모두 종료/제출 시 COMPLETED
- 시간 초과 시 DELAYED

PASS 기준

- 상태 판정 로직이 문서와 100% 일치

10-9. syncSchedulePhotoFlags Step 분해

Step 1

photo metadata 조회

Step 2

BEFORE/AFTER 존재 계산

Step 3

schedule flag update

Step 4

테스트

- before만 있는 경우
- after만 있는 경우
- 둘 다 있는 경우

PASS 기준

- hasBeforePhotos / hasAfterPhotos 정확

---

11. 함수별 Step 분해표 — 장비/NFC 도메인

11-1. createEquipment Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증

Step 3

equipmentCode 중복 검사

Step 4

기본 payload 생성

- status = AVAILABLE
- isActive 기본값 처리

Step 5

write 수행

Step 6

audit log 생성

Step 7

테스트

- 정상 등록
- 코드 중복 실패

PASS 기준

- 장비 생성
- 기본 상태 AVAILABLE

11-2. updateEquipmentMeta Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증

Step 3

상태 필드 수정 금지 validator 구현

Step 4

허용 필드만 update

Step 5

audit log 생성

Step 6

테스트

- 메타 수정 성공
- status 수정 시도 실패

PASS 기준

- status/currentHolder/time 필드 보호

11-3. assignNfcTag Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증

Step 3

equipment 조회 및 활성 검증

Step 4

기존 ACTIVE mapping 조회

Step 5

nfcTagId 중복 ACTIVE mapping 조회

Step 6

교체 정책 구현

- 기존 mapping 비활성 전환 여부

Step 7

트랜잭션 구현

- mapping upsert
- equipment.nfcTagId 반영
- 필요 시 equipment_logs 생성
- audit log 생성

Step 8

테스트

- 신규 매핑
- 기존 교체
- 중복 ACTIVE 실패

PASS 기준

- 1:1 ACTIVE 관계 유지

11-4. checkoutEquipment Step 분해

Step 1

입력 타입 정의

Step 2

인증/활성 사용자 확인

Step 3

nfcTagId 정규화 유틸 적용

Step 4

mapping 조회

Step 5

equipment 조회

Step 6

사전 검증

- isActive
- status AVAILABLE
- mapping 일치

Step 7

트랜잭션 재조회

- 최신 status 재확인

Step 8

write builder 구현

- status CHECKED_OUT
- holder 설정
- lastCheckoutAt 기록

Step 9

equipment_logs payload 생성

Step 10

audit_logs payload 생성

Step 11

커밋

Step 12

테스트

- 정상 반출
- 태그 미등록 실패
- 이미 반출중 실패
- 동시성 충돌 실패

PASS 기준

- 장비 상태/보유자/시간/로그 동시 반영
- partial write 0

11-5. checkinEquipment Step 분해

Step 1

입력 타입 정의

Step 2

인증/활성 확인

Step 3

mapping + equipment 조회

Step 4

status ∈ {CHECKED_OUT, OVERDUE} 검증

Step 5

대리 반입 정책 반영

Step 6

트랜잭션 구현

- status AVAILABLE
- holder 제거
- lastCheckinAt 기록
- equipment_logs 생성
- audit_logs 생성

Step 7

테스트

- 정상 반입
- 이미 AVAILABLE 실패
- 태그 불일치 실패

PASS 기준

- holder 제거 포함 완전 복귀

11-6. forceUpdateEquipmentStatus Step 분해

Step 1

입력 타입 정의

Step 2

관리자 인증

Step 3

사유 필수 validator

Step 4

허용 예외 전이 검증

Step 5

필요 시 holder 제거

Step 6

equipment_logs + audit_logs 생성

Step 7

테스트

- AVAILABLE→DISABLED
- CHECKED_OUT→DISABLED
- DISABLED→AVAILABLE

PASS 기준

- reason 없는 강제 변경 불가

11-7. markEquipmentOverdueBatch Step 분해

Step 1

thresholdHours 입력 처리

Step 2

CHECKED_OUT 장비 조회

Step 3

초과 여부 판정기 구현

Step 4

OVERDUE 갱신 배치 처리

Step 5

필요 시 로그 생성

Step 6

테스트

- 24시간 초과 장비 전환
- 미초과 장비 유지

PASS 기준

- 대상만 OVERDUE로 정확히 전환

---

12. 함수별 Step 분해표 — 정기청소/운영 도메인

12-1. createRegularCleaningAttendanceBatch Step 분해

- site 검증
- workerIds 검증
- 중복 attendance 조회
- batch 생성
- audit log 생성

PASS 기준

- 같은 날짜/장소/사원 중복 생성 금지

12-2. recordRegularCleaningStart Step 분해

- 본인 attendance 검증
- startAt 기록

PASS 기준

- 타인 attendance 입력 차단

12-3. recordRegularCleaningEnd Step 분해

- startAt 존재 확인
- endAt > startAt
- 종료 기록

PASS 기준

- 역시간 차단

12-4. approveRegularCleaningAttendance Step 분해

- 관리자 인증
- attendance 조회
- participationApproved 반영
- adminAdjustmentNote 기록

PASS 기준

- 관리자만 승인 가능

12-5. createDownloadLog Step 분해

- 관리자 인증
- exportType 검증
- download_logs 생성

PASS 기준

- 로그 append only

12-6. recalcDashboardMetrics Step 분해

- 기간 기준 입력 처리
- KPI source readers 구현
- cache write

PASS 기준

- 오늘 KPI와 대시보드 카드 값이 일치

12-7. detectDelayedSchedulesBatch Step 분해

- 진행중/예정 일정 조회
- expectedEndTime 초과 여부 계산
- DELAYED 전환

PASS 기준

- 취소/완료 일정 오탐 없음

12-8. cleanupDrafts Step 분해

- draft retention 정책 확인
- 만료 draft 조회
- 삭제 또는 비활성 처리

PASS 기준

- 운영 중 문서 손상 없음

---

13. 함수별 PASS / FAIL 기준 (통합)

PASS 공통 기준

- 인증 검증 존재
- 활성 사용자 검증 존재
- role 검증 존재
- 입력 validator 존재
- 상태 전이 검증 존재
- 필요한 트랜잭션 적용
- 필요한 로그 생성
- Emulator 테스트 통과
- 컴파일 에러 0

- any 타입 없음

FAIL 공통 기준

- 클라이언트 입력값을 그대로 상태에 반영
- 로그 없는 상태 변경
- 부분 커밋 발생 가능
- 중복 호출 방어 없음
- 잘못된 상태 전이 허용
- 타인 데이터 접근 허용
- 테스트 없이 배포 후보 처리

---

14. 실제 개발 착수 순서 (하루 단위 운영 기준)

Day 1

- functions 프로젝트 골격 생성
- shared 공통부 생성
- index.ts export 구조 생성
- types 디렉토리 생성

Day 2

- createScheduleWithWorkers
- updateScheduleAndWorkers
- cancelSchedule
- emulator 테스트

Day 3

- recordWorkStart
- recordWorkEnd
- recomputeScheduleStatus
- 테스트

Day 4

- syncSchedulePhotoFlags
- submitClosing
- reviewClosing
- 테스트

Day 5

- createEquipment
- updateEquipmentMeta
- assignNfcTag
- 테스트

Day 6

- checkoutEquipment
- checkinEquipment
- forceUpdateEquipmentStatus
- 테스트

Day 7

- markEquipmentOverdueBatch
- 정기청소 함수들
- 운영 보조 함수 일부

주의: 이것은 하루 묶음 예시일 뿐, 실제 에이전트 작업은 항상 **15분 초원자 단위**로 쪼개야 한다.

---

15. 개발 시작 전 최종 체크리스트

-

---

16. 개발 시작 선언

이 문서가 준비된 시점부터는 더 이상 "무엇을 만들지"를 고민하는 단계가 아니다.

이제부터는 아래 순서로 바로 실행한다.

1. Shared 공통부 생성
2. 일정 생성/배정 Functions 구현
3. 사원 시작/종료/마감 Functions 구현
4. 장비/NFC Functions 구현
5. 정기청소/운영 보조 Functions 구현
6. Emulator 테스트
7. 그 다음 UI 연결 시작

즉, **Functions 구현 우선순위 + 파일 구조 + 함수별 Step 분해표는 여기서 확정**이며, 이 문서를 기준으로 바로 개발에 들어간다.

---

17. 최종 선언

이 문서는 HELLA OPS Functions 개발의 **실행 우선순위 SSOT**다.

앞으로 Functions 구현 시 다음은 금지한다.

- 우선순위 무시한 임의 구현
- 파일 구조 혼합

- 한 파일에 여러 도메인 비즈니스 로직 혼합
- 상태 전이 검증 없는 빠른 구현
- 트랜잭션 필요한 기능의 임시 분리 구현
- 테스트 없는 완료 보고

이 문서의 목적은 단순 정리가 아니라, **Antigravity와 AI 에이전트가 표류 없이 즉시 개발을 시작하도록 실행 경로를 고정하는 것**이다.